

Tron Game

Yusupov Boburjon

Neptun: YTAJDI

December 6, 2025

1 Description of the Exercise

Implement a Tron-like game where two players control light cycles (motors) on a 2D grid. The game is played on a 60x40 grid. Player 1 (Red) starts on the left side moving right, and Player 2 (Blue) starts on the right side moving left. As players move, they leave a trail behind them. If a player collides with a wall, an obstacle, or any existing trail (including their own), they crash and the other player wins. If both players crash simultaneously, it is a draw.

The game must include dynamic obstacles generated for each level. The number of obstacles increases with the level difficulty. The game must also track player scores (number of wins) and store them in a persistent database (PostgreSQL). The application should provide a main menu to enter player names, choose colors, start the game, and view high scores.

2 Class Diagram

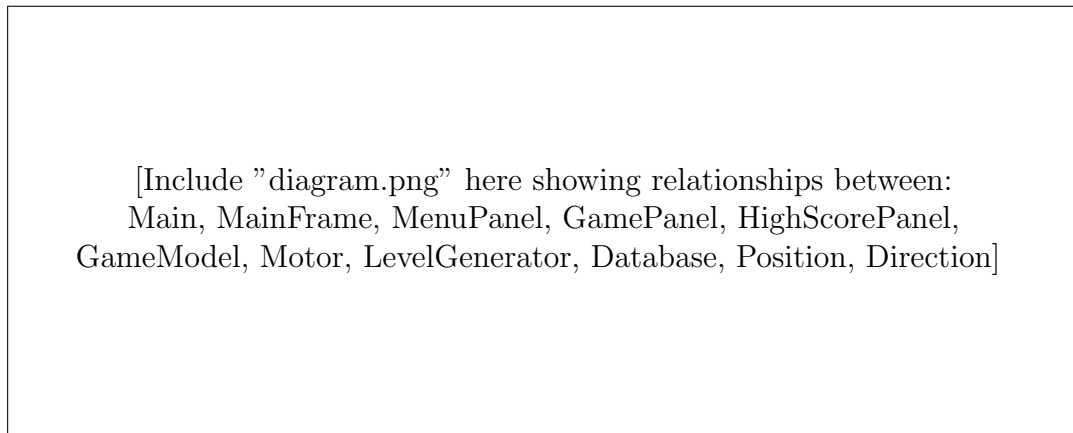


Figure 1: Class Diagram of the Tron Application

2.1 Class Overview

The application follows the Model-View-Controller (MVC) architectural pattern:

- **View (GUI):** Classes in the 'tron.gui' package (MainFrame, MenuPanel, GamePanel, HighScorePanel).
- **Model (Logic):** Classes in the 'tron.logic' package (GameModel, Motor, LevelGenerator, Position, Direction).
- **Data (DB):** Classes in the 'tron.db' package (Database, PlayerScore).
- **Controller:** Event handling is managed within the View classes (ActionListeners) delegating to the Model.

3 Class and Method Descriptions

3.1 Main Class (`tron.Main`)

Entry point for the application.

- `main(String[] args)`: Launches the application by creating the `MainFrame` on the Event Dispatch Thread.

3.2 Game Mechanics (`tron.logic`)

3.2.1 `GameModel`

Manages the core state of the game session.

- `update()`: Advances the game state by one tick. Moves players and checks for collisions.
- `checkCollisions()`: Determines if any player has crashed into walls, obstacles, or trails.
- `checkCrash(Motor player)`: detailed collision logic for a single player.
- `getWinnerName()`: Returns the name of the winner or "Draw".

3.2.2 `Motor`

Represents a player's entity.

- `move()`: Updates the motor's position based on its current direction and adds the new position to its trail.
- `setDirection(Direction newDirection)`: Changes the player's direction, preventing 180-degree turns.

3.2.3 `LevelGenerator`

Handles procedural generation of proper game levels.

- `generateLevel(int levelIndex)`: Creates a list of random `Positions` for obstacles. Ensures obstacles do not spawn in player starting safety zones.

3.3 Database Integration (`tron.db`)

3.3.1 `Database`

Handles JDBC connections to PostgreSQL.

- `addWin(String playerName)`: Increments the win count for a player in the 'results' table. Uses an 'INSERT ... ON CONFLICT DO UPDATE' statement.
- `getHighScores()`: Queries the top 10 players ordered by score.

3.4 User Interface (`tron.gui`)

3.4.1 MainFrame

The main container managing the application screens using ‘CardLayout’.

- `showMenu()`: Switches to start screen.
- `startGame(...)`: Switches to the game view and initializes a new session.
- `showHighScores()`: Switches to the high score table.

3.4.2 GamePanel

Renders the game loop.

- `actionPerformed(ActionEvent e)`: The game loop tick handler.
- `paintComponent(Graphics g)`: Draws the grid, players, trails, and obstacles.

4 Test Cases

Unit tests are implemented using JUnit 4.

4.1 Logic Layer Tests

4.1.1 GameModelTest

- **testInitialization**: Verifies players start at correct positions.
- **testUpdateMovesPlayers**: checks that ‘update()’ advances player coordinates.
- **testWallCollision**: Simulates a player hitting the wall and verifies game over state and winner designation.
- **testHeadOnCollision**: Simulates players crashing into each other, resulting in a draw.

4.1.2 MotorTest

- **testInitialization**: Checks initial state of a motor.
- **testMove**: Verifies movement logic and trail growth.
- **testDirectionChange**: checks valid 90-degree turns.
- **testInvalidDirectionChange**: checks that 180-degree turns are ignored.

4.1.3 LevelGeneratorTest

- **testGenerateLevelCount**: Verifies that the number of obstacles scales with level index.
- **testSafetyZones**: Ensures no obstacles are generated in the corners where players spawn.

4.1.4 DirectionTest

- **testEnumValues**: Basic verification of Direction enum constants.

4.1.5 PositionTest

- **testPositionValues**: checks record accessor methods.
- **testEqualsAndHashCode**: Verifies value-based equality for positions.

4.2 Test Summary

Test Class	Number of Tests
GameModelTest	4
MotorTest	4
LevelGeneratorTest	2
DirectionTest	1
PositionTest	2
Total	13

Table 1: Test Coverage Summary